



Today's agenda

- ↳ Insert delete get Random $O(1)$
- ↳ Insert delete get Random $O(1)$ - duplicates
- ↳ Random Pick with Blacklist.



AlgoPrep



// How to get random number in a range.

$l[0, 10] \rightarrow 11 \rightarrow 2/11$
 $0 \rightarrow 1 \dots 10$

$\rightarrow$ Random $r = \text{new Random}();$

$\rightarrow r.\text{nextInt}(2, 11);$
 $[2, 10]$

Q) $\{1-100\}$ & $\{1000-1100\} \rightarrow \text{get random.}$

// idea 1

$\hookrightarrow r.\text{nextInt}(1, 1101);$ $\xrightarrow{200} 250$

// idea 2

$0 \ 1 \ 2 \ 3 \ 4 \ 5 \dots 150 \dots 200$
 $1 \dots 100, 100, 100 \dots 1100$

$\rightarrow r.\text{nextInt}(0, 201); \rightarrow 150$



Q) Insert delete get Random O(1)

↳ implement the class:

a) bool insert(val) → insert val in set if not Present. Return true if item was not Present.

b) bool remove(val) → Remove an item val from set if Present. return true if item was Present.

c) int getRandom() → returns a random element from the current set of elements. Each element must have the same Probability.

ex: add(10) add(12) add(14) add(16) get() remove(12) remove(10)

map (no.) Integer Integer (ion in list)

14

10	-80
12	-81
14	-80
16	-81

list	0	1	2
14	16	10	

remainder = 0



// Pseudo code

Class RandomizedSet {

HashMap < Integer, Integer > hm;

ArrayList < Integer > list;

Random r;

Public RandomizedSet () {

hm = new HashMap < > ();

list = new ArrayList < > ();

}

boolean insert (int val) {

if (hm.containsKey (val) == true) {

return false;

}

else {

hm.put (val, list.size());

list.add (val);

return true;

}

}

$O(1)$ ←

int getRandom () {

int id = r.nextInt (list.size());

return list.get (id);

}

$O(1)$ ←



```
boolean remove (int val) {
```

```
    if (hm.containsKey(val) == false) { return false; }
```

```
    int idn = hm.get(val);
```

```
    hm.remove(val);
```

```
    if (idn == list.size() - 1) { list.remove(list.size() - 1);  
                                return true; }
```

$O(1)$ ←

```
    int idn2 = list.size() - 1;
```

```
    int temp = list.get(idn2);
```

```
    swap(idn, idn2);
```

```
    list.remove(list.size() - 1);
```

```
    hm.put(temp, idn);
```

```
    return true;
```

```
}
```

3



```
boolean remove (int val) {  
    if (hm.containsKey(val) == false) {return false;}  
    int idn = hm.get(val);  
    hm.remove(val);  
    int idn2 = list.size() - 1;  
    int temp = list.get(idn2);  
    swap(idn, idn2);  
    list.remove(list.size() - 1);  
    hm.put(temp, idn);  
    return true;  
}
```

list
0 1 2
~~30~~ 20 ~~30~~

int idn = hm.get(val);
hm.remove(val);

int idn2 = list.size() - 1;
int temp = list.get(idn2);
swap(idn, idn2);
list.remove(list.size() - 1);

hm.put(temp, idn);
return true;

~~10 - 0~~
20 - 1
30 - ~~2~~ 0

idn = 0

idn2 = 2

temp = 30



AlgoPrep



Q) Insert delete get Random $O(1)$ → duplicates allowed

add(10) add(12) add(14) add(10) add(12) add(14) add(10)

add(12) gR() → $\text{rand}(\text{0}, \text{list.size()-1})$ remove(10)

remove(14)

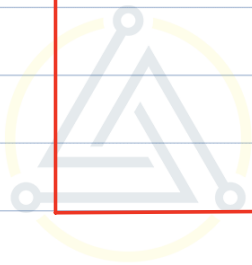
Integer → val

HashSet → indices

10	-	3 2
12	-	1 4 0
14	-	5

list: 0 1 2 3 4 5
~~12~~ 12 ~~10~~ 10 12 14 ~~14~~

idx = 2



AlgoPrep



//Pseudo code

```
HashMap<Integer, HashSet<Integer>> hm;  
ArrayList<Integer> list;  
Random r;
```

$O(1)$ ←

```
boolean insert (int val) {  
    HashSet<Integer> set;  
    boolean flag;  
    if (hm.containsKey(val) == true) {  
        set = hm.get(val);  
        flag = false;  
    } else {  
        set = new HashSet<>();  
        flag = true;  
    }  
    set.add(list.size());  
    hm.put(val, set);  
    list.add(val);  
    return flag;  
}
```

$O(1)$ ←

```
int getRandom() {  
    int id = r.nextInt(list.size());  
    return list.get(id);  
}
```




```
boolean remove (int val) {
```

```
    HashSet<Integer> set;
```

```
    if (hm.containsKey(val) == true) {
```

```
        set = hm.get(val);
```

```
    }
```

```
    else {
```

```
        return false;
```

```
    }
```

```
    int remidn = -1;
```

```
    for (int n : set) {
```

```
        remidn = n;
```

```
        break;
```

```
    }
```

```
    set.remove(remidn);
```

```
    map.put(val, set);
```

```
    if (remidn == list.size() - 1) {
```

```
        list.remove(list.size() - 1);
```

```
    }
```

```
    else {
```

```
        int idn2 = list.size() - 1;
```

```
        int temp = list.get(idn2);
```

```
        swap(remidn, idn2);
```

```
        list.remove(list.size() - 1);
```

```
        HashSet<Integer> s = map.get(temp);
```

```
        s.remove(idn2);
```

```
        s.add(remidn);
```

```
        map.put(temp, s);
```

```
    }
```

$O(1)$ ←

3

if (map.get(val).size() == 0) {map.remove(val);
return true;



add(10) add(12) add(14) add(10)

```
boolean insert (int val) {
    HashSet<Integer> set;
    boolean flag;
    if (hm.containsKey(val) == true) {
        set = hm.get(val);
        flag = false;
    }
    else {
        set = new HashSet<>();
        flag = true;
    }
    set.add(list.size());
    map.put(val, set);
    list.add(val);
    return flag;
}
```

hm

10	-	0 3
12	-	1
14	-	2

list

0	1	2	3
10	12	14	10

3

```
boolean remove (int val) {
    HashSet<Integer> set;
    if (hm.containsKey(val) == true) {
        set = hm.get(val);
    }
    else {
        return false;
    }
    int remidn = -1;
    for (int n : set) {
        remidn = n;
        break;
    }
    set.remove(remidn);
    map.put(val, set);
}
```

add(10) add(12) add(14) add(10)

hm

10	-	0 2
12	-	1
14	-	0

list

0	1	2
10	12	10

remove(10)

```
if (remidn == list.size() - 1) {
    list.remove(list.size() - 1);
}
else {
    int idn2 = list.size() - 1;
    int temp = list.get(idn2);
    swap(remidn, idn2);
    list.remove(list.size() - 1);
    HashSet<Integer> s = map.get(temp);
    s.remove(idn2);
    s.add(remidn);
    map.put(temp, s);
}
if (map.get(val).size() == 0) {map.remove(val);
return true;
```

remidn: ~~0~~

temp: 14

idn2: 3

3



Q) Random Pick with Blacklist

↳ Return a random Integer in the range $[0, N-1]$ and not in blacklist.

Ex: $N = 20$ $BL: \{1, 3, 7, 13, 17, 19\}$

0

//idea 1

1

↳ get random in $[0, N-1]$ and return only if it is not present in Blacklist.

2

3

4

5

//idea 2

6

↳ Add numbers from $[0, N-1]$ except the blacklisted numbers and get random on index.

7

8

9

10

T.C: $O(N)$

S.C: $O(N)$

11

12

13

14

15

16

17

18

19

$n = 20$

BL: { 1 3 7 13 17 19 } 

0

1

valid numbers count = $20 - 5 = 15$

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

→ Do random in $[0, n-m)$

5

7

3

16

18

AlgoPrep

valid

2



// Pseudo code

```
HashMap<Integer, Integer> map;
```

```
Random r;
```

```
int valid;
```

```
Public Solution (int n, int[] blacklist) {
```

```
    map = new HashMap<>();
```

```
    r = new Random();
```

```
    for (int i = 0; i < blacklist.length; i++) {
```

```
        map.put(blacklist[i], -1);
```

```
    }
```

```
    valid = n - blacklist.length;
```

T.C: $O(n)$

```
    for (int i = 0; i < blacklist.length; i++) {
```

```
        if (blacklist[i] < valid) {
```

```
            while (map.containsKey(n-1) == true) n--;
```

```
            map.put(blacklist[i], n);
```

```
            n--;
```

```
        }
```

```
    }
```

```
}
```

```
int Pick () {
```

```
    int val = r.nextInt(valid);
```

```
    if (map.containsKey(val) == true) { return map.get(val); }
```

```
    else { return val; }
```

```
}
```



$N = 20$
~~18~~
~~17~~
~~16~~
15

BL: { 3 7 13 17 19 }

valid = $20 - 5 = 15$

```
for (int i=0; i< blacklist.length; i++) {  
    map.put(blacklist[i], -1);  
}
```

```
int valid = N - blacklist.length;
```

```
for (int i=0; i< blacklist.length; i++) {  
    if (blacklist[i] < valid) {  
        while (map.containsKey(N-i) == true) { N--; }  
        map.put(blacklist[i], N-i);  
        N--;  
    }  
}
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

3 → ~~18~~

7 → ~~16~~

13 → ~~15~~

17 → -1

19 → -1



AlgoPrep